

软件工程基础第 5 章

软件需求分析

详细学习笔记

中南大学计算机学院任胜兵

笔记整理

2026 年 5 月 28 日

目录

1 需求分析概述	2
1.1 需求分析的概念	2
1.2 需求分析的特点	2
2 需求收集 (Requirements Collection)	2
2.1 需求收集的过程	2
2.2 需求收集的内容	3
2.3 需求收集的方式	3
3 数据流建模 (Data Flow Modeling)	3
3.1 数据流建模概述	3
3.2 数据流图 (Data Flow Diagram, DFD)	4
3.2.1 数据流图的基本组成元素	4
3.2.2 数据流图示例说明	4
3.3 分层数据流图 (Hierarchical DFD)	5
3.3.1 为什么需要分层数据流图	5
3.3.2 顶层数据流图 (Context Diagram)	5
3.3.3 底层数据流图 (Primitive DFD)	6
3.3.4 中间层数据流图 (Intermediate DFD)	6
3.4 数据词典 (Data Dictionary, DD)	6
3.4.1 数据词典的概念	6
3.4.2 数据词典卡片示例	7
3.5 加工说明 (Process Specification)	8
3.5.1 加工说明的概念	8
3.5.2 IPO 图 (Input/Process/Output)	8

3.5.3	结构化语言 (Structured Language)	9
3.5.4	判定表与判定树 (Decision Table and Decision Tree)	9
3.6	数据流建模步骤	9
4	IDEF0 功能建模 (IDEF0 Functional Modeling)	10
4.1	IDEF 方法概述	10
4.2	IDEF0 图的组成	10
4.2.1	盒子 (Box) 和箭头 (Arrow)	10
4.2.2	盒子的四边含义	10
4.2.3	箭头的种类	11
4.3	分层 IDEF0 图	11
4.4	IDEF0 建模步骤	12
5	IDEF1X 数据建模 (IDEF1X Data Modeling)	12
5.1	IDEF1X 概述	12
5.2	IDEF1X 图的核心概念	12
5.2.1	实体 (Entity)	12
5.2.2	联系 (Relationship)	13
5.2.3	属性 (Attribute)	13
5.3	IDEF1X 建模步骤	14
6	UML 建模语言 (Unified Modeling Language)	14
6.1	UML 概述	14
6.1.1	UML 的定义与起源	14
6.1.2	UML 的结构	15
6.2	UML 的 4+1 视图构架	15
6.3	UML 的关系标记法和语义	16
6.4	UML 的图概览	16
6.5	静态模型图详解	17
6.5.1	类图 (Class Diagram)	17
6.5.2	包图 (Package Diagram)	17
6.5.3	组件图 (Component Diagram)	18
6.5.4	部署图 (Deployment Diagram)	18
6.5.5	对象图 (Object Diagram)	18
6.6	动态模型图详解	19
6.6.1	用例图 (Use Case Diagram)	19
6.6.2	顺序图 (Sequence Diagram)	19
6.6.3	协作图 (Collaboration Diagram)	20
6.6.4	状态图 (Statechart Diagram)	20
6.6.5	活动图 (Activity Diagram)	21
6.7	UML 的特点	21

6.8 UML 的应用	22
7 用例建模 (Use Case Modeling)	22
7.1 用例建模概述	22
7.2 用例图的组成	22
7.3 参与者及用例的描述	23
7.3.1 参与者描述	23
7.3.2 用例描述方法	24
7.4 用例与需求的关系	24
7.5 用例建模步骤	24
8 对象建模 (Object Modeling)	25
8.1 对象建模概述	25
8.2 确定对象与类——对象层 (The Object Layer)	25
8.2.1 对象与类的概念	25
8.2.2 确定对象/类的方法	26
8.2.3 电梯控制系统的对象/类标识示例	27
8.3 标识结构——结构层 (The Structure Layer)	27
8.3.1 结构层的概念	27
8.3.2 标识结构的方法	28
8.3.3 电梯控制系统的结构标识示例	28
8.4 标识主题——主题层 (The Subject Layer)	28
8.4.1 主题层的概念	28
8.4.2 标识主题的方法	29
8.4.3 电梯控制系统的主题层	29
8.5 定义属性及实例关联——属性层 (The Attribute Layer)	29
8.5.1 属性层的概念	30
8.5.2 实例关联的 UML 表示	30
8.5.3 属性定义	30
8.5.4 实例关联定义	31
8.5.5 电梯控制系统的属性层	31
8.6 定义服务及消息连接——服务层 (The Service Layer)	32
8.6.1 服务层的概念	32
8.6.2 定义消息连接	33
8.6.3 电梯控制系统的服务层	33
9 需求规格说明与验证 (Requirements Specification & Verification)	34
9.1 需求规格说明概述	34
9.2 需求规格说明方法	34
9.3 需求验证	35
9.3.1 需求验证的重要性	35

9.3.2 有效需求规格说明的特征	35
9.3.3 需求验证完成后的注意事项	35
10 小结 (Summary)	35
10.1 本章核心要点	35
10.2 本章知识体系图	36

1 需求分析概述

1.1 需求分析的概念

[p.2]

需求分析（Requirements Analysis）是指开发人员通过细致的调查分析，详细、准确和完整地理解用户需要什么样的软件，将用户非形式的需求陈述转化为完整的需求定义，再将需求定义转换到相应的需求规格说明的过程。

通常，把一整套的需求分析方法、技术和工具等的集合称为**建模方法**（Modeling Method）。

需求分析的核心任务可以概括为：

- (1) 将用户非形式的需求陈述转化为完整的需求定义
- (2) 再将需求定义转换到相应的需求规格说明（Software Requirements Specification, SRS）

1.2 需求分析的特点

[p.3]

需求分析面临着诸多困难和挑战，主要包括以下几个方面：

1. **问题的复杂性**：软件系统本身复杂，涉及多个领域和业务流程。
2. **交流障碍**：开发人员和用户之间存在沟通鸿沟，用户可能不明白技术术语，开发人员可能不了解业务领域。正如常说的——“我知道你相信你理解了我所说的，但我不能确定你是否认识到你听到的并不是我所意指的。”
3. **需求易变性**：用户需求在开发过程中可能不断变化和调整。
4. **不一致性和不完整性**：不同用户可能提出相互矛盾的需求，或者需求描述不完整。

解决方案（Solution）：

- 开发人员必须与有关人员密切配合，充分交换意见
- 借助各种建模方法
- 对用户提出的各项要求，开发人员应认真分析，不能机械地全盘接受

2 需求收集 (Requirements Collection)

2.1 需求收集的过程

[p.4]

需求收集（Requirements Elicitation）是需求分析的第一个阶段，主要工作包括：

- (1) 同各种用户进行交流
- (2) 收集各种用户的信息

- (3) 理解用户的各项要求
- (4) 对信息进行分析
- (5) 澄清一些模糊的要求
- (6) 向用户解释不合理的或暂时无法实现的要求

2.2 需求收集的内容

[p.5]

需求收集需要关注以下五个方面的需求：

表 1: 需求收集的五大内容

需求类别	说明
信息需求（数据需求）	系统需要处理哪些数据，数据的格式、结构、存储方式等
功能需求	系统需要完成哪些功能，提供哪些服务
性能需求	系统在响应时间、吞吐量、资源利用率等方面的要求
运行需求	系统运行所需的硬件环境、软件环境、网络环境等
未来需求	系统未来可能的扩展和升级方向

此外，还包括系统交付需求、标准需求、实现方法需求、资源使用需求等其它方面的需求。

2.3 需求收集的方式

[p.6]

常用的需求收集方式包括：

- (1) 访谈（Interview）：包括程式化的访谈（Structured Interview）和非程式化的访谈（Unstructured Interview）。程式化的访谈有固定的问题列表，非程式化的访谈则更灵活。
- (2) 问卷调查（Questionnaire Survey）：通过书面问卷向大量用户收集需求和意见。
- (3) 场景使用（Scenario Analysis）：通过描述典型使用场景来发现和确认需求。
- (4) 用户资料收集（User Documentation Collection）：收集现有的用户手册、报表、表格等资料来分析需求。

3 数据流建模 (Data Flow Modeling)

3.1 数据流建模概述

[p.7]

数据流建模方法（Data Flow Modeling）是一种结构化分析方法（Structured Analysis, SA），其核心特点包括：

- 自顶向下、逐层分解地定义系统需求
- 利用数据流图（Data Flow Diagram, DFD）来对用户需求进行分析
- 可用于分析任何应用系统的需求

结构化分析方法的精髓在于：从整体到局部，从抽象到具体，逐步细化系统的功能和数据流动。

3.2 数据流图 (Data Flow Diagram, DFD)

[p.8]

3.2.1 数据流图的基本组成元素

数据流图由四种基本元素组成：

表 2: 数据流图的四种基本元素

元素名称	表示符号	说明
数据流（Data Flow）	箭头 →	表示数据在系统中的流动方向，箭头所指方向为数据流向
加工（Process/Bubble）	圆圈或圆角方框	表示对数据进行的处理和变换，将输入数据流变换为输出数据流
数据存储（Data Store）	开口矩形框或双划线	表示数据的静态存储，如文件、数据库等
数据的源点和终点（Source/Sink）	正方形或立方体	表示系统外部的实体，即数据的产生者或接收者

扩展符号主要有：*（与）、+（或）和 ⊕（异或）等。

3.2.2 数据流图示例说明

在教材 p.8 的示例中，展示了一个仓库管理系统的数据流图：

- 仓库管理员（源点）产生“更新库存事务”数据流
- “处理事务”加工（圆圈 1）接收该数据流，访问数据存储 D1（库存清单），产生输出到数据存储 D2（定货信息）
- “产生报表”加工（圆圈 2）从 D2 读取数据，生成“定货报表”输出给采购员（终点）

3.3 分层数据流图 (Hierarchical DFD)

[p.9-12]

3.3.1 为什么需要分层数据流图

[p.9]

只用一张数据流图来描述整个系统存在两个主要问题：

- (1) 难以一次画齐所有细节
- (2) 即使画齐了也难于理解

分层数据流图的优势：

- 避免一次引入过多的细节，有利于控制问题的复杂度
- 便于对大型系统描述的实现
- 不同的用户可以只选择分层数据流图中与本身有关或感兴趣的部分，不必阅读全图，便于用户的使用和理解

3.3.2 顶层数据流图 (Context Diagram)

[p.10]

顶层数据流图（也称上下文图，Context Diagram）主要描述整个系统的作用范围，说明系统的边界，反映系统和外部环境之间的关系，即系统的输入和输出数据流。

关键特征：

- 顶层数据流图**只有一张**
- 将整个系统表示为一个加工（一个圆或方框）
- 只画出系统与外部实体（源点和终点）之间的数据流

教材示例——考试事务管理系统：

- 外部实体：考生、阅卷点、考试中心
- 输入数据流：报名表（来自考生）
- 输出数据流：考生名单、考生通知单（到阅卷点）、准考证（到考生）、成绩表（到考生）、统计报表（到考试中心）
- 输入控制：合格标准（来自考试中心）

3.3.3 底层数据流图 (Primitive DFD)

[p.11]

底层数据流图由一些不必再进行分解的加工组成，这些加工称为基本加工（Primitive Process）。

教材示例（p.11 图 1）展示了报名登记功能被分解为三个基本加工：

1. 检查报名表（加工 1.1）：检查报名表是否合格，输出不合格报名表（返回给考生）或合格报名表
2. 编号（加工 1.2）：对合格报名表进行编号，生成已编号报名表
3. 登记（加工 1.3）：将已编号报名表信息登记，生成准考证输出给考生，同时将考生名单存储到考生花名册

3.3.4 中间层数据流图 (Intermediate DFD)

[p.12]

中间层数据流图是通过分解高层加工得到的，其中有些加工还需进一步分解。它位于顶层和底层之间，起到过渡和细化的作用。

教材示例（p.12 图 0）展示了考试事务管理系统被分解为两个主要加工：

1. 报名登记：处理报名表，产生准考证和考生名单
2. 成绩统计：处理成绩表，产生考生通知单和统计报表

两者共享数据存储 F1（考生花名册）。

3.4 数据词典 (Data Dictionary, DD)

[p.13-14]

3.4.1 数据词典的概念

[p.13]

数据词典（Data Dictionary，又称数据字典）是关于数据信息的集合，是对数据流图中的每个数据（包括数据流和数据存储）进行严格定义的场所，以保持数据在系统中的一致性。

数据词典的作用：

- 定义数据流图中每个数据元素的精确含义
- 保持数据在整个系统中的一致性
- 为开发团队提供统一的数据术语参考

3.4.2 数据词典卡片示例

[p.14]

数据词典通过卡片形式定义各种数据。常见的数据词典卡片包括：

(a) 数据流卡片——示例“考生名单”：

- 名称：考生名单
- 描述：供阅卷点使用的考生信息表
- 组成：{考生编号 + 考生姓名}

(b) 数据项卡片——示例“考生编号”：

- 名称：考生编号
- 别名：考号、准考证号
- 描述：考生的统一编号
- 取值及含义：YYXXXX（YY 表示参加考试的年份 2 位数字，XXXX 表示顺序号 4 位数字）
- 注释：顺序号从 0001 开始至 9999

(c) 数据存储卡片——示例“考生花名册”：

- 名称：考生花名册
- 描述：保存考生的个人信息
- 组成：{考生编号 + 考生姓名 + 通讯地址 + [电话 | 电子邮箱] + (照片)}
- 组织方式：按考生编号从小到大排列
- 注释：照片为彩色，1 寸大小

(d) 扩展的数据词典卡片——示例“查询”：

- 名称：查询
- 描述：查询顾客、存货和发票的有关信息
- 组成：[顾客状态查询 | 存货查询 | 发票存根查询]
- 数据量：2000 次/天
- 峰值：每天上午 9:00—10:00 有 1000 次
- 注释：到 2005 年底预计增加 3 到 5 种查询

组成描述符说明：

- {}: 表示重复（多个）
- | : 表示选择（或）
- (): 表示可选
- +: 表示与（和）

3.5 加工说明 (Process Specification)

[p.15-19]

3.5.1 加工说明的概念

[p.15]

数据流图中的“基本加工”由于没有进一步分解得到子图，因而需要**加工说明**来对其进行描述。

加工说明的核心内容：

- 描述基本加工如何把输入数据流变换成输出数据流的**加工规则**
- 描述实现加工的**策略**而不是实现加工的细节

常用的加工说明工具包括：**IPO 图**、**结构化语言**、**判定表**、**判定树**等。

3.5.2 IPO 图 (Input/Process/Output)

[p.16]

IPO 图 (Input/Process/Output) 是一种描述加工逻辑的图形工具，由三个部分组成：

- **输入框** (Input)：列出加工所需的输入数据
- **处理框** (Process)：描述加工的处理逻辑
- **输出框** (Output)：列出加工产生的输出数据

教材示例：

输入	处理	输出
1. 主文件	1. 校验主记录	1. 有效主记录
2. 事务文件	2. 校验事务记录	2. 有效事务记录
	3. 更新主记录	3. 更新后的主文件

IPO 图除可用于分析阶段描述加工逻辑说明外，也常与层次图 (Hierarchy Chart, HC) 一起用于设计阶段，形成 **HIPO 图** (Hierarchy plus Input-Process-Output)。

3.5.3 结构化语言 (Structured Language)

[p.17]

结构化语言又称 PDL (Program Design Language, 程序设计语言) 或**伪代码** (Pseudo Code), 是一种介于自然语言和形式语言之间的半形式语言。

特点:

- 使用有限的词汇和固定的句式
- 语法结构清晰, 类似程序语言
- 比自然语言更精确, 比形式语言更易理解

教材示例——发货批准逻辑:

```
IF 发货单金额 > 1万元人民币 THEN
    IF 账上欠款时间 > 30天 THEN
        在偿还欠款前不批准
    ELSE
        发批准书及发货单
    ENDIF
ENDIF
```

3.5.4 判定表与判定树 (Decision Table and Decision Tree)

[p.18-19]

判定表 (Decision Table) 和 **判定树 (Decision Tree)** 适合描述以下情况: 完成加工的一组动作是由于某一组条件取值的组合而引发的。

判定树是判定表的**图形表示**, 有时比判定表更直观。

判定树示例——水费计算 (p.19):

	季度用水 < 160t	按固定比率计算
固定比率		
	季度用水 160t	按表 中的比率计算
计算水费		
	季度用水 < 160t	按表 中的比率计算
可变比率		
	季度用水 160t	按表 中的比率计算

3.6 数据流建模步骤

[p.20]

数据流建模的步骤原则上是由外向里、自顶向下去模拟问题的处理过程:

步骤 1: 画顶层数据流图: 确定系统边界, 识别外部实体和主要输入/输出数据流

步骤 2：画分层数据流图：自顶向下逐层分解加工，直到达到基本加工

步骤 3：用数据词典定义数据流图中的所有数据：包括数据流、数据存储和数据项

步骤 4：用加工说明描述数据流图中的基本加工：使用 IPO 图、结构化语言、判定表/判定树等工具

4 IDEF0 功能建模 (IDEF0 Functional Modeling)

4.1 IDEF 方法概述

[p.21]

1981 年美国空军 ICAM (Integrated Computer Aided Manufacturing) 工程中使用了名为“IDEF” (ICAM DEFinition method) 的方法，后来称之为 Integration DEFinition method。

IDEF 方法族：

- **IDEF0**：进行功能建模 (Functional Modeling)
- **IDEF1X**：建立数据模型 (Data Modeling)
- **IDEF4**：面向对象设计 (Object-Oriented Design)
- 此外还有 IDEF2 (仿真建模)、IDEF3 (过程描述获取)、IDEF5 (本体论描述获取) 等

IDEF0 方法用严格的自顶向下、逐层分解的方式来构造系统的功能模型，用 IDEF0 图来描述。

4.2 IDEF0 图的组成

[p.22-23]

4.2.1 盒子 (Box) 和箭头 (Arrow)

[p.22]

IDEF0 图的主要元素是简单的盒子及箭头：

- **盒子**：代表系统的功能 (活动)
- **箭头**：表示系统处理的数据约束，可以是具体的事物，也可以是抽象的信息

IDEF0 图有时也称为活动图形 (Activity Diagram)。

4.2.2 盒子的四边含义

[p.23]

每个盒子有四条边线，分别对应四种数据约束：

表 3: IDEF0 盒子四边含义

边线位置	英文	含义与说明
输入边线（左）	Input	表示功能需要消耗或转化的数据/事物（What），如坯料（原材料）
输出边线（右）	Output	表示功能产生的结果（What），如成品零件
控制边线（上）	Control	表示约束或指导功能执行的条件（Why），如蓝图、工单、调度表
机制边线（下）	Mechanism	表示执行功能所需的手段或资源（How），如刀具、机器、样板

关键规则： IDEF0 功能建模方法要求一张 IDEF0 图中的盒子最多只能有 **6 个**（子图形还要求不少于 **3 个**）。

4.2.3 箭头的种类

[p.24-29]

(1) 分支箭头 [p.24]：表示多个活动需要同一种数据或同一种数据的不同成分。

重要说明： 在 IDEF0 图中，箭头代表**数据约束**，而不是数据流或执行顺序。在同一图上，若几个盒子所需的数据约束都满足，则这几个活动可**同时执行**。

(2) 汇合箭头 [p.25]：表示多个活动产生（或合成）同一种数据。

(3) 通道箭头 [p.26]：表示箭头描述的数据约束不出现在子图（或父图）中。通道箭头可使子图的描述得到简化，也可避免在高层父图中出现信息过细的现象，干扰用户对图形的理解。

(4) 双向箭头 [p.27]：表示两个盒子描述的活动互为输入或互为控制，且将先触发的盒子画在较高的位置上。

(5) 虚线箭头 [p.28]：表示活动触发的先后顺序（时序关系）。

(6) 选择箭头 [p.29]：表示数据的选择关系（逻辑“或”关系），可使用与普通箭头不同的标记方式。

4.3 分层 IDEF0 图

[p.30]

为了明确分解过程中数据和图形间的关系，IDEF0 采用 **ICOM 码**和**结点号**来进行标识。

ICOM 码：对于来自父盒子的数据约束，分别用 I、C、O、M 表明来自父盒子的输入（Input）、控制（Control）、输出（Output）及机制（Mechanism）边线，并在字母之后跟上数据约束在父盒子中的数字顺序号（编号顺序为从上至下、从左至右）。

结点号：用来表明图形或盒子在分层 IDEF0 图中的位置。结点号采用树形结构编号，例如：

- A-0：顶层图（描述系统与外部环境的关系）
- A0：第 0 层图（系统的第一级分解）
- A1, A2, A3, ...：第 1 层各图

- A21, A22, A23, ...: 第 2 层各图 (A2 的子图)

文字说明编号规则 [p.32]:

- 图形的文字说明编号为: 图形结点号 + 字母 “T” (例如 A12T)
- 图形的其它参考图形编号为: 图形结点号 + 字母 “F” + 顺序号 (例如 A1F4)

4.4 IDEF0 建模步骤

[p.34]

步骤 1: 确定建模的范围、观点及目的: 明确建模的边界和视角

步骤 2: 建立系统的内外关系图, 即 A-0 图: 描述系统与外部环境的关系 (顶层图)

步骤 3: 建立 A0 图的一系列子图: 自顶向下逐层分解

步骤 4: 书写文字说明: 对每个图形添加文字解释

5 IDEF1X 数据建模 (IDEF1X Data Modeling)

5.1 IDEF1X 概述

[p.35]

IDEF1X 方法是 IDEF1 方法的扩展版本:

- **IDEF1**: 用来表示系统的信息结构和语义
- **IDEF1X**: 增强了图形的表达能力, 丰富了语义和简化了开发过程

IDEF1X 本质上是一种实体关系建模 (Entity-Relationship Modeling) 方法, 但比传统的 ER 图更丰富和精确。

5.2 IDEF1X 图的核心概念

[p.36-41]

5.2.1 实体 (Entity)

[p.36]

实体是具有相同属性或特征的现实或抽象事物的集合, 这个集合中的一个元素称为实体的一个实例 (Instance)。

关键规则: 在一张 IDEF1X 图中, 一个实体只能在图中出现一次。

实体分为两类:

- **独立实体 (Independent Entity)**: 不依赖于其它实体而存在的实体。用直角方框表示。例如: 供应商

- **从属实体 (Dependent Entity)**: 依赖于其它实体而存在的实体。用圆角方框表示。例如: 供应合同

实体的标识格式为: 实体名/实体编号 (如: 供应商/2)。

5.2.2 联系 (Relationship)

[p.37-40]

IDEF1X 中的联系分为两大类:

一、确定联系 (Specific Relationship):

(1) 连接联系 (Connection Relationship) [p.38]:

- **可标定联系 (Identifying Relationship)**: 子实体的实例唯一地由父实体确定, 子实体的主键包含父实体的主键。在图中用实线表示。
- **非标定联系 (Non-Identifying Relationship)**: 子实体的实例不唯一地由父实体确定, 父实体的主键作为子实体的非主键属性。在图中用虚线表示。

连接联系的**基数 (Cardinality)**: 一对一、一对多。

(2) 分类联系 (Categorization Relationship) [p.39]:

- **完全分类联系 (Complete Categorization)**: 子类实体完全覆盖父类实体的所有实例, 每个父类实例必须是某个子类的实例。
- **不完全分类联系 (Incomplete Categorization)**: 子类实体不完全覆盖父类实体的所有实例。

二、非确定联系 (Non-Specific Relationship) [p.40]:

多对多关系: 两个实体之间的多对多关系。在 IDEF1X 中, 非确定联系需要转化为确定联系。转化方法为: 引入一个**中间实体 (Associative Entity)**, 将多对多关系分解为两个一对多关系。

教材示例——学期与课程: 学期和课程之间是多对多关系 (一个学期开设多门课程, 一门课程可在多个学期开设)。通过引入中间实体“提供”(提供/3), 转化为:

- 学期/1 对提供/3: 一对多
- 课程/2 对提供/3: 一对多

5.2.3 属性 (Attribute)

[p.41]

属性表示现实或抽象事物的特性或性质。

属性的分类:

- **关键字属性 (Primary Key)**: 唯一标识实体实例的属性或属性组合。在图中放在实体框内的上方, 用横线与其它属性分隔。

- **次关键字属性** (Alternate Key, AK): 也可以用做标识但未被选为主键的属性。
- **一般属性** (Non-Key Attribute): 描述实体特征的非键属性。

教材示例——职员实体:

- **关键字**: 职员编号
- **次关键字**: 身份编号 (AK1)、姓名 (AK2)、生日 (AK2)…… (AK 后面的数字表示次键的编号)

5.3 IDEF1X 建模步骤

[p.42]

IDEF1X 建模分为五个阶段:

表 4: IDEF1X 建模的五个阶段

阶段	主要任务
阶段 0: 确定建模目标和计划	明确建模的范围、目的和计划
阶段 1: 定义实体	识别系统中的所有实体
阶段 2: 定义联系	识别实体之间的各种联系 (连接联系、分类联系等)
阶段 3: 定义键	为每个实体确定主键和次键
阶段 4: 定义属性	为每个实体定义所有非键属性

6 UML 建模语言 (Unified Modeling Language)

6.1 UML 概述

[p.44-46]

6.1.1 UML 的定义与起源

[p.44-45]

UML (Unified Modeling Language, 统一建模语言) 是一种可以应用于任何面向对象软件开发方法的标记法和语义语言。

UML 的融合过程 [p.45]: UML 融合了三种主流面向对象方法的概念:

- **Grady Booch 的 Booch 方法**: 强调设计和构造
- **James Rumbaugh 的 OMT 方法** (Object Modeling Technique): 强调分析和数据模型
- **Ivar Jacobson 的 OOSE 方法** (Object-Oriented Software Engineering): 强调用例驱动

正如 Grady Booch 所说: “如果你有好的思想, 那它也是我们的。” (If you have a good idea, it's ours too.)

6.1.2 UML 的结构

[p.46]

UML 的组成包括以下几个方面：

(1) 构造块（Building Blocks）——UML 的基本建模元素：

- 物件（建模元素）：
 - 结构物件（Structural Things）：如类、接口、协作、用例、组件、节点等
 - 行为物件（Behavioral Things）：如交互、状态机等
 - 分组物件（Grouping Things）：如包（Package）
 - 注释物件（Annotational Things）：如注释（Note）
- 关系（Relationships）——物件间的联系：
 - 关联（Association）
 - 依赖（Dependency）
 - 泛化（Generalization）
 - 实现（Realization）
- 图（Diagrams）——表示模型的可视化方式

(2) 公共机制（Common Mechanisms）：

- 规格说明（Specification）：图中各元素语义的文字描述
- 修饰（Adornment）：增强图的整体清晰性和可读性
- 公共划分（Common Divisions）：两种划分方法——类元/实例（Classifier/Instance）和接口/实现（Interface/Implementation）
- 扩展机制（Extensibility Mechanisms） [p.49]:
 - 约束（Constraint）：用花括号 { } 括起的文本字符串，说明必须维持为真的条件和规则
 - 构造型（Stereotype）：已有模型元素的变体，用书名号 《 》 括起，如《interface》、《include》
 - 标记值（Tagged Value）：用花括号括起的标记值列表，格式为 {tag1=value1, tag2=value2, ...}

(3) 构架（Architecture）——4+1 视图 [p.50]:

6.2 UML 的 4+1 视图构架

[p.50]

UML 采用“4+1”视图来描述系统架构：

表 5: UML 的 4+1 视图构架

视图	包含的图	面向对象与关注点
用例视图（Use Case View）	用例图、活动图、顺序图、协作图	最终用户、分析员和测试人员；描述系统行为、用户和系统的交互
逻辑视图（Logical View）	类图、状态图、包图、对象图	分析设计人员；描述系统的面向对象模型、功能和结构
实现视图（Implementation View）	组件图	编程人员；描述源代码和软件组件的组织
进程视图（Process View）	类图、对象图	集成人员；描述系统的并发和同步机制，包括进程、线程
部署视图（Deployment View）	部署图	系统工程师；描述硬件拓扑结构和组件的物理分布

6.3 UML 的关系标记法和语义

[p.47]
UML 中四种基本关系的标记法和语义：

表 6: UML 四种关系的标记法和语义

关系类型	标记法	语义说明
关联（Association）	实线（可带箭头）	描述类实例之间的连接，表示一个类对象与另一个类对象存在某种语义联系。可以有名称、角色、多重性和导航方向
依赖（Dependency）	虚线带箭头	表示一个建模元素的变化会影响另一个建模元素（使用关系），但反之不成立
泛化（Generalization）	实线带空心三角箭头	描述类之间的“is-a”关系，即一般类（父类）和特殊类（子类）之间的继承关系
实现（Realization）	虚线带空心三角箭头	表示类实现了接口，即一个类保证履行另一个类（或接口）定义的契约

6.4 UML 的图概览

[p.48]
UML 提供了丰富的图形类型，分为两大类：

一、静态模型图（Structural Diagrams）：描述系统的静态结构

- 类图（Class Diagram）
- 包图（Package Diagram）

- 组件图 (Component Diagram)
- 部署图 (Deployment Diagram)
- 对象图 (Object Diagram)

一、动态模型图 (Behavioral Diagrams): 描述系统的动态行为

- 用例图 (Use Case Diagram)
- 顺序图 (Sequence Diagram)
- 协作图 (Collaboration Diagram)
- 状态图 (Statechart Diagram)
- 活动图 (Activity Diagram)

6.5 静态模型图详解

6.5.1 类图 (Class Diagram)

[p.51–52]

类图 (Class Diagram) 是 UML 中最核心的静态图，用于描述系统的静态结构：

- 描述了系统的**静态特性**
- 描述了**对象的结构**
- 将行为实体描述成“离散”的模型元素，不包括动态行为细节
- 关键元素是**类元 (Classifier)**，如类、接口等以及它们之间的关系
- 类元之间有关联 (Association)、泛化 (Generalization) 及各种依赖关系 (Dependency)，包括实现 (Realization) 和使用 (Usage) 关系

类的表示法：类用包含三个栏的矩形表示：

- **名称栏 (顶部)：**类的名称
- **属性栏 (中部)：**类的属性，格式为 可见性属性名：类型 = 默认值 (可选)
- **操作栏 (底部)：**类的操作/方法，格式为 可见性操作名 (参数列表)：返回类型 (可选)

类图示例说明 [p.52]：类图中示出了类名、属性、操作，以及附加信号信息。

6.5.2 包图 (Package Diagram)

[p.53]

包图 (Package Diagram) 由包或类组成，表示包与包、包与类之间的关系，用于描述系统的分层结构。

包图中的**包**用文件夹图形来描述 (类似文件系统中的文件夹图标)。

教材示例——保险系统包图：

- 内部系统包包含：数据库界面包、保险单填写界面包
- 数据库界面包包含：保险单包、客户包
- 保险单包依赖于 Oracle 界面和 Sybase 界面

包图主要用于：

- 组织大型系统的类，降低模型复杂度
- 描述系统的分层架构
- 表示子系统之间的依赖关系

6.5.3 组件图 (Component Diagram)

[p.54]

组件图 (Component Diagram, 又称构件图), 用来描述软件的各个组件 (包括源代码文件、二进制文件、脚本、可执行文件等) 之间的**依赖关系**。

在 UML 中, 组件使用左侧带两个小矩形的大矩形表示。组件之间使用虚线箭头表示依赖关系。

教材示例：展示 ImageObserver 接口和 component.java 之间的依赖关系。组件可以包含接口 (用构造型 «interface» 标识)。

6.5.4 部署图 (Deployment Diagram)

[p.55]

部署图 (Deployment Diagram, 又称配置图), 显示的是对**运行时处理节点**以及其中的组件的配置, 反映了系统硬件的**物理拓扑结构**。

在 UML 中, 节点用一个**三维矩形**表示。

教材示例——保险系统部署图：

- 客户 PC 节点：包含保单填写界面组件
- 保险服务器节点：包含保险系统、保险数据库、保险系统配置组件
- 节点之间通过 «TCP/IP» 通信协议连接

6.5.5 对象图 (Object Diagram)

[p.56]

对象图 (Object Diagram) 描述一组对象以及它们之间的关系。

对象是类的一个具体**实例** (Instance), 在 UML 中采用矩形表示, 其中标明对象名和属性取值。

对象名的表示格式：对象名：类名 (如 “李冰: Author”)。

教材示例——李冰的电脑：

- 李冰: Author 对象, name="李冰", age=28
- 李冰的台式电脑: Computer 对象, name="联想 1+1", supplier="联想集团"
- 李冰的手提电脑: Computer 对象, name="ThinkPad", supplier="IBM"
- 李冰“使用”两台电脑（关联关系）

对象图可以看作是类图在某一时刻的快照（Snapshot）。

6.6 动态模型图详解

6.6.1 用例图 (Use Case Diagram)

[p.57]

用例图（Use Case Diagram）从系统外部执行者的角度来描述系统需要提供哪些功能，指明这些功能的参与者，即用例图描述了参与者和用例及它们之间的关系。

用例图的主要组成元素：

- **参与者**（Actor）：用小人符号表示，代表与系统交互的外部实体（用户、其他系统等）
- **用例**（Use Case）：用椭圆表示，代表系统提供的功能单元
- **系统边界**（System Boundary）：用方框表示，界定系统的范围
- **关系**：参与者与用例之间的通信关系（连线），用例之间的关系（包含、扩展、泛化）

教材示例——简单 ATM 系统用例图：

- **参与者**：顾客（主要参与者）、银行系统（次要参与者）、管理员（次要参与者）
- **用例**：取款、存款、转账（顾客使用）、维护（管理员使用）

6.6.2 顺序图 (Sequence Diagram)

[p.58]

顺序图（Sequence Diagram）用来建模以时间顺序安排的对象间的交互。

顺序图的组成：

- **对象生命线**（Lifeline）：垂直虚线，表示对象存在的时间
- **激活条**（Activation Bar）：细长矩形，表示对象正在执行操作的时间段
- **消息**（Message）：对象之间的箭头，表示操作调用
 - **同步消息**（Synchronous Message）：实心三角箭头，发送者等待接收者完成
 - **异步消息**（Asynchronous Message）：开放箭头，发送者不等待
 - **返回消息**（Return Message）：虚线箭头
- **控制权**（Focus of Control）：表示对象拥有控制权的时间段

教材示例——汽车诊断与维修顺序图：

- Mechanic（机修工）向 Diagnosis（诊断系统）和 Car（汽车）发送消息
- 消息流：Turn On（打开）→ Check Diagnosis（检查诊断）→ Diagnose（诊断）→ Repair Car（修车）→ Turn Off（关闭）

6.6.3 协作图 (Collaboration Diagram)

[p.59]

协作图（Collaboration Diagram，又称合作图/通信图），用来建模对象或角色之间的交互，描述这些对象或角色之间是如何彼此通信的。

协作图与顺序图的对比：

- 顺序图强调时间顺序（纵向时间轴）
- 协作图强调对象间的结构关系（空间布局）
- 两者在语义上是等价的，可以相互转换

教材示例——汽车诊断协作图（与 p.58 的顺序图对应）：消息编号：1:TurnOn, 2:TurnOn, 3:CheckDiagnosis, 4:Diagnose, 5:RepairCar, 6:TurnOff, 7:TurnOff（编号表示消息发送的顺序）。

6.6.4 状态图 (Statechart Diagram)

[p.60]

状态图（Statechart Diagram）描述一个类对象所经历的各种状态以及事件发生时状态的转移条件。

状态图的组成元素：

- **状态**（State）：圆角矩形，表示对象在生命周期中的某个条件或状况
- **转移**（Transition）：带箭头的线，表示从一种状态到另一种状态的变化，标注事件名称
- **初始状态**（Initial State）：实心圆点
- **终止状态**（Final State）：实心圆点外加圆圈

教材示例——电梯状态图：

- 主要状态：空闲（Idle）、下降（Moving Down）、上行（Moving Up）
- 达到第一层（到达事件）触发从“下降”到“空闲”的转移
- 超时事件触发状态变化
- 到达顶层触发从“上行”到“下降”的转移

6.6.5 活动图 (Activity Diagram)

[p.61-62]

活动图 (Activity Diagram) 是由状态图变化而来的, 描述需要执行的活动以及执行这些活动的顺序。

活动图的组成:

- **活动节点 (Activity Node):** 圆角矩形, 表示一个活动/动作
- **控制流 (Control Flow):** 实线箭头, 表示活动之间的执行顺序
- **起始节点 (Initial Node):** 实心圆
- **终止节点 (Final Node):** 实心圆加圆圈
- **分支/合并 (Decision/Merge):** 菱形, 表示条件分支
- **分叉/汇合 (Fork/Join):** 粗横线, 表示并行活动

教材示例——咖啡炉活动图 [p.61]:

开机器 → 开动 → [调制咖啡/信号灯灭] → 倒咖啡 → 咖啡炉

泳道 (Swimlane) [p.62]: 活动图可使用泳道来划分不同角色/部门负责的活动。泳道形式的活动图用垂直或水平分隔线将活动分配到不同的参与者。

教材示例——销售过程泳道图:

- **顾客泳道:** 请求服务 → 支付 → 收集
- **售货员泳道:** 开订单
- **库房泳道:** 交货 → 供货

6.7 UML 的特点

[p.63]

UML 具有以下三个主要特点:

- (1) **统一了面向对象方法的基本概念:** UML 融合了 G. Booch 的 Booch 方法、J. Rumbaugh 的 OMT 方法和 I. Jacobson 的 OOSE 方法中的有关概念, 消除了不同方法之间的术语差异和表示差异。
- (2) **具有更强的建模能力:** 正如 G. Booch 所说: “如果你有好的思想, 那它也是我们的。” UML 集成了多种方法的优点, 形成了一套全面的建模语言。
- (3) **独立于特定的开发语言和开发过程:** UML 不是一种方法学, 而是一种建模语言, 可以与不同的开发过程 (如 RUP、敏捷等) 和编程语言配合使用。

6.8 UML 的应用

[p.64]

UML 在不同开发阶段的应用：

表 7: UML 在软件开发各阶段的应用

开发阶段	使用的 UML 图
需求分析	用例图：描述功能需求；类图：描述静态结构；状态图、顺序图和协作图：描述类之间所需的协作，实现用例
设计	定义软件系统中的技术细节用到的类，如引入处理用户交互的类、处理数据的类、处理通信和并行性的类等
实现	组件图：代码组件的物理结构及组件之间的关系；部署图：硬件的拓扑结构和组件的分布
测试	类图：单元测试；组件图、协作图：集成测试；用例图：确认测试

针对 UML 的应用, Rational Software 公司提出了**统一过程** (Rational Unified Process, RUP) 方法 [p.64–65]。

RUP 的核心工作流 [p.65] 包括：业务建模、需求、分析与设计、实现、测试、部署、配置与变更管理、项目管理、环境等。

7 用例建模 (Use Case Modeling)

7.1 用例建模概述

[p.66]

用例建模 (Use Case Modeling) 是一种从用户使用系统的角度来建立系统功能需求模型的方法。

用例建模的特点：

- 既非从数据模型开始，亦非从系统数据流着手
- 而是从组成系统的实际操作入手
- 以用户视角为中心，聚焦于“系统能为用户做什么”

用例图 (Use Case Diagram, 又称用例模型) 是用例建模的主要成果，它从系统外部执行者的角度来描述系统需要提供哪些功能以及谁使用这些功能。

7.2 用例图的组成

[p.67]

用例图主要有**四种基本成分**：

- (1) **系统 (System)**：用方框表示，界定系统的边界。系统名称标注在方框顶部。

(2) 参与者 (Actor): 用小人图标表示。分为:

- 主要参与者 (Primary Actor): 主动发起用例执行的参与者
- 次要参与者 (Secondary Actor): 被用例影响或参与用例执行的参与者

(3) 用例 (Use Case): 用椭圆表示, 描述系统提供的一项完整功能, 从用户视角提供有价值的结果。

(4) 关系 (Relationship): 参与者与用例之间、用例与用例之间的各种关系:

- 通信关系 (Communication): 参与者与用例之间的连线, 表示参与者参与该用例
- 包含关系 (Include): 用《include》标识, 虚线箭头指向被包含的用例。表示一个用例的行为总是包含另一个用例的行为
- 扩展关系 (Extend): 用《extend》标识, 虚线箭头指向被扩展的基础用例。表示在特定条件下, 一个用例的行为可以扩展到另一个用例的行为中
- 泛化关系 (Generalization): 参与者之间或用例之间的继承关系, 子参与者或用例继承父参与者或用例的行为

教材示例——ATM 系统用例图 [p.67]:

- 系统: 简单 ATM 系统
- 主要参与者: 顾客 (使用取款、存款、转账功能)
- 次要参与者: 管理员 (使用维护功能)、银行系统
- 包含关系: 取款《include》验证顾客、存款《include》验证顾客
- 扩展关系: 转账《extend》通知透支 (条件: [顾客订购了“透支通知”服务且操作引起透支])
- 泛化关系: 管理支取 (父用例) 与取款 (子用例) 之间

7.3 参与者及用例的描述

[p.68-73]

7.3.1 参与者描述

[p.68]

参与者描述的内容主要包括:

- 参与者的名称
- 是否为抽象参与者
- 对参与者的简要描述 (角色、职责等)

7.3.2 用例描述方法

[p.69–73]

用例描述有许多种方法，开发人员可根据项目进展及用户特点灵活选择：

- 方法 1：简单文字描述** [p.69]：使用自然语言简要描述用例的功能。例如：用例“获取呼叫历史”使客户可以查阅账上已付费的所有呼叫的细节。呼叫历史可以以文本或声音的形式提供给客户。
- 方法 2：模板描述** [p.70]：使用标准化的模板来详细描述用例，模板通常包含用例名称、参与者、前置条件、后置条件、基本事件流、备选事件流等字段。
- 方法 3：表格描述** [p.71]：使用表格形式组织用例信息，便于快速查看和比较。
- 方法 4：图形描述** [p.72]：使用活动图、顺序图等图形来描述用例的执行流程。
- 方法 5：形式化语言描述** [p.73]：使用形式化语言（如 Z 语言）进行严格精确的用例描述。Z 语言是一种基于集合论和一阶谓词逻辑的形式化规格说明语言。

7.4 用例与需求的关系

[p.74]

用例是捕获功能需求的强大工具。用例与需求之间存在对应关系：

- **功能需求**：直接映射为用例
- **非功能需求**：作为用例的补充说明或约束条件
- 用例提供了一种结构化的方式来组织和优先级排序需求

7.5 用例建模步骤

[p.75]

用例建模遵循以下步骤：

步骤 1：找出系统的参与者和用例：

- **选择参与者的两个标准**：(1) 应该能至少确定一个用户来扮演参与者；(2) 与系统相关的不同参与者实例所充当的角色间的重叠应该最少
- **确定用例的两个准则**：(1) “有价值的结果”——用例应为参与者提供可感知的价值；(2) “特定参与者”——每个用例应明确指定一个主要参与者

步骤 2：区分用例的优先次序：根据重要性和紧急程度排列用例的开发顺序

步骤 3：详细描述每个用例：使用适当的方法（文字、模板、表格、图形或形式化语言）描述用例的详细内容

步骤 4：构造用户界面原型：基于用例创建界面原型，帮助用户直观理解系统功能

步骤 5：构造用例图：综合上述内容，绘制完整的用例图

注意事项：

- 用例的结构关系应尽量反映真实情况
- 需要将每个单独的用例视为一个单独的制品
- 应尽量避免从功能上分解用例（即不要按功能模块拆分用例，应保持用例的完整性）

8 对象建模 (Object Modeling)

8.1 对象建模概述

[p.76]

对象建模 (Object Modeling) 的主要任务是了解某个特定应用问题域内所涉及的对象，以及各种各样的结构和通信关系。

教材采用 **Coad/Yourdon** 面向对象分析方法来进行对象建模，在表示符号上采用 UML 建模语言提供的标记法。

Coad/Yourdon 方法的五个层次：

- (1) 对象层 (Object Layer)
- (2) 结构层 (Structure Layer)
- (3) 主题层 (Subject Layer)
- (4) 属性层 (Attribute Layer)
- (5) 服务层 (Service Layer)

8.2 确定对象与类——对象层 (The Object Layer)

[p.77-80]

8.2.1 对象与类的概念

[p.77]

- **对象 (Object)**：是现实世界某些事物的一个抽象，它反映该事物在系统中需要保存的信息和发挥作用。
- **类 (Class)**：是具有相同属性 (Attribute) 和操作 (Operation) 的一组对象的集合，是对一组具有相同特征的对象抽象描述。

UML 中类与对象的表示法：

类的表示：

- **名称栏**（顶部）：类名。可选修饰如 {author=Ren, status=tested}（标记值）
- **属性栏**（中部）：可见性属性名：类型 = 默认值（可选）
 - 可见性：+（public，公有的）、#（protected，受保护的）、-（private，私有的）
- **操作栏**（底部）：可见性操作名（参数）：返回类型（可选）

对象的表示：对象名：类名，如下划线表示是实例。

教材示例：

- Rectangle 类：属性和操作栏被压缩显示，带有标记值 {author=Ren, status=tested}
- mingLi: Teacher 对象：名称栏带有下划线，属性栏有具体值

8.2.2 确定对象/类的方法

[p.78]

有三种主要的方法用于标识对象和类：

方法 1：三视图模型法（Three-View Model）：

- 实体-关系模型（Entity-Relationship Model, ER Model）
- 数据流模型（Data Flow Model）
- 状态-迁移模型（State-Transition Model）

通过这三个视角来全面识别系统中的对象。

方法 2：基于语言的信息分析法（Linguistic-Based Information Analysis）：

- **短语频率分析（Phrase Frequency Analysis）：**从需求文档中提取名词短语，统计频率，高频名词可能是对象候选
- **矩阵分析（Matrix Analysis）：**通过对象之间的交叉矩阵来分析

方法 3：CRC 卡片法（Class-Responsibility-Collaboration Cards）：

- CRC 卡片是一种简单而有效的面向对象分析技术
- 每张卡片代表一个类，卡片内容包括：
 - 类名（Class Name）
 - 职责（Responsibilities）：该类需要完成的任务
 - 协作类（Collaborators）：为完成职责而需要交互的其他类
- 教材示例 BankAccount CRC 卡片：职责——维护账户平衡；协作类——Bank

8.2.3 电梯控制系统的对象/类标识示例

[p.79-80]

问题描述 [p.79]: 电梯控制系统用以调度和控制一幢 50 层高的建筑物中的 5 部电梯。电梯控制系统必须高效而合理地调度电梯。在没有召唤和未完成的目的地请求时, 电梯就应停在最后所到达的楼层, 直到再次投入使用。同时, 在将乘客送往目的地之前不能反方向运行。而且, 一个已经满载的电梯 (每部电梯均有超载传感器可供系统程序查询了解是否超载) 不再受理新的召唤请求。

识别出的对象/类 [p.80]:

- 电梯 (Elevator)
- 超载传感器 (Overload Sensor)
- 召唤请求面板 (Summon Panel)
- 到达事件 (Arrival Event)
- 到达面板 (Arrival Panel)
- 目的地事件 (Destination Event)
- 目的地面板 (Destination Panel)
- 召唤事件 (Summon Event)
- 楼层 (Floor)

特别说明: 将“楼层”作为一个候选的对象/类, 主要用来管理电梯的分派和到达, 并管理有关召唤请求的信息。

重要提示:

- (1) 最后得到的对象/类不可能被证明是绝对正确或绝对错误——对象建模具有一定的艺术性和主观性
- (2) 用户的偏好对模型最后所选择的对象/类有很大的影响——不同的利益相关者可能强调不同的对象

8.3 标识结构——结构层 (The Structure Layer)

[p.81-83]

8.3.1 结构层的概念

[p.81]

标识结构是用来处理对象建模复杂性的机制之一。现实世界的事物之间有两种主要关系:

(1) 继承关系 (Inheritance / Generalization-Specialization):

- 又称泛化-特化关系 (“is-a” 关系)
- 一般类 (General Class) 和特殊类 (Special Class) 之间的关系
- UML 表示法: 实线 + 空心三角箭头, 箭头指向一般类
- 教材图示: (a) 继承关系 < 一般类 > ◁ < 特殊类 >

(2) 整体/部分关系 (Whole-Part Relationship):

- **聚合关系**（Aggregation）（“has-a” 关系）：整体和部分之间具有独立的生命周期，部分可以脱离整体而存在
 - UML 表示法：(b) 空心菱形在整体端，实线连接
- **组合关系**（Composition）（“contains-a” 关系，强聚合）：整体和部分之间具有相同的生命周期，部分不能脱离整体而独立存在
 - UML 表示法：(c) 实心菱形在整体端，实线连接

8.3.2 标识结构的方法

[p.82]

对于分类继承关系：

- **自顶向下方式**：从一般类出发，寻找其特殊类（特化）
- **自底向上方式**：从特殊类出发，提取共同特征形成一般类（泛化）

对于整体/部分关系：

- 把每一个对象的类看成是一个**整体**，考虑：什么是它的部分或组成成分？
- 把每一个对象的类看成是一个**部件**，考虑：该对象的类适合什么样的组装整体？

8.3.3 电梯控制系统的结构标识示例

[p.83]

识别出的继承关系：

- 面板（一般类）→ 目的地面板、到达地板板（特殊类）
- 电梯事件（一般类）→ 到达事件、召唤事件、目的地事件（特殊类）

识别出的整体/部分关系：

- 电梯（整体）◇ 电梯马达（部分）、超载传感器（部分）
- 楼层（整体）◇ 召唤面板（部分）

8.4 标识主题——主题层 (The Subject Layer)

[p.84-86]

8.4.1 主题层的概念

[p.84]

在对象建模的过程中，大多数模型都是非层次式的。**Coad/Yourdon** 方法使用主题（Subject）来处理规模比较大的复杂模型。

主题的作用：

- 控制一个用户必须同时考虑的模型数目
- 有助于了解模型的总体概貌
- 提供一种分组机制，将语义上相关的建模元素组织在一起

在 Coad/Yourdon 方法中，主题采用方框来描述，并对每个主题进行编号。在 UML 中，包（Package）可用来把语义上相关的建模元素分组为内聚的单元，因而也可用来表示主题。

8.4.2 标识主题的方法

[p.85]

- (1) 对每一个结构增加一个相应的主题
- (2) 对每个对象增加一个相应的主题
- (3) 将紧密耦合的主题组合起来得到更好的主题
- (4) 如果主题的数目较多，不符合“**7±2 规则**”（人类的短期记忆一般能处理 5-9 个信息块），则应进一步精炼主题

注意事项：

- 在选择主题时，不应用功能分解方法
- 开发人员可以依据子问题域、子系统，甚至组织或地域来区分主题
- 若将一个主题看成是问题域中的整体/部分关系，即问题域由主题组成，则对建立模型很有用

8.4.3 电梯控制系统的主题层

[p.86]

将电梯控制系统的对象/类划分为两个主题：

主题 1： 电梯管理主题：包含“电梯”、“电梯马达”、“超载传感器”、“到达面板”、“目的地面板”和“面板”，主要用于控制硬件。

主题 2： 电梯调度主题：包含“电梯事件”、“到达事件”、“召唤事件”、“目的地事件”、“楼层”和“召唤面板”，主要检测事件的发生和对电梯进行相应的调度。

注：由于电梯控制系统的规模并不是很大，因而也可不必引入主题层。

8.5 定义属性及实例关联——属性层 (The Attribute Layer)

[p.87-91]

8.5.1 属性层的概念

[p.87]

属性（Attribute）是对象/类的数据单元，定义属性使得问题域更加明确，也使对对象/类和结构的认识更深入具体。

在分析阶段的基本原则：

- 一般不使用属性来表示对象/类之间的关系
- 而使用对象/类的**实例关联**（Instance Association）来描述对象/类之间的关系
- 实例关联可以看成是一种事务规则或应用论域限制
- 当实现这些对象的时候，这些事务规则指明操作如何运行，以确保与系统的策略相一致

8.5.2 实例关联的 UML 表示

[p.88]

在 UML 中，实例的关联表示包括：

- **关联名称**（Association Name）：描述关联含义的名字
- **角色名称**（Role Name）：描述一个类在关联中扮演的角色
- **多重性**（Multiplicity）：每个类参与关联的实例数量
 - 格式：**minimum..maximum**（如 0..1, 1..*, 1, *）
 - 如果多重性不显式说明，则多重性不确定
- **导航性**（Navigability）：表示消息能否沿箭头所指的方向传递
 - 有箭头：单向导航
 - 无箭头：双向导航（默认）

注意事项：

- 一般，在图中要么注明关联名称，要么注明角色名称，而不同时注明角色名称和关联名称
- 教材示例：Company 类和 Person 类之间的 employs 关联，角色名为 employer/employee，多重性为 1 对 *

8.5.3 属性定义

[p.89]

属性的种类：

- (1) **描述性属性**（Descriptive Attribute）：描述对象自身的特征，如“学生”的“身高”
- (2) **命名性属性**（Naming Attribute）：用于标识或命名对象，如“学生”的“姓名”、“学号”

- (3) **参考属性** (Referential Attribute)：引用其它对象的属性，如“学生”的“系别”（引用“系”对象）

定义属性的原则和方法：

- 属性的确定既与问题域有关，也与目标系统的任务有关
- 定义属性应该在**原子概念的层次**上进行（不可再分解）
- 依据属性的种类考察每个对象/类可帮助定义属性
- 如果已建立数据视图（如 IDEF1X 图、ER 图），数据视图中的实体的属性表示了存储数据的需求，这些需求必须体现在模型中，一般作为对象/类的属性
- 通过考察对象的描述，也可为定义属性提供帮助
- **对于分类结构，应该利用继承机制来定位属性的位置：**将共有属性放在一般类中，特有属性放在特殊类中

8.5.4 实例关联定义

[p.90]

实例关联反映了对象/类的实例之间的相互依赖、相互作用的关系。整体/部分关系是一种特殊的实例关联。

定义实例关联的方法：

- 直接提取需求陈述中的**动词词组**
- 利用领域知识
- 矩阵分析

实例关联的选择原则：应仅选择那些系统必须维持的实例对应关系。

教材示例——车主与交通工具：

- 原始多对多关系：车主 *——拥有——* 交通工具（带有拥有标识、日期修饰）
- 转化为：引入“执照”作为关联类 (Associative Class)，车主 1——执照 *，执照 *——交通工具 1，执照包含执照号和日期属性

8.5.5 电梯控制系统的属性层

[p.91]

主要属性定义：

- **电梯：**标识号、当前行进方向、最近到达楼层、分派后应运行方向、状态
- **电梯马达：**标识号
- **超载传感器：**状态

- 面板：标识号
- 到达面板：标识号
- 目的地面板：标识号、目的地楼层 [1..50]
- 召唤面板：标识号、上行状态、下行状态
- 电梯事件：标识号
- 到达事件：电梯标识号、到达楼层
- 目的地事件：电梯标识号、目的地楼层
- 召唤事件：召唤类型、召唤楼层
- 楼层：标识号

实例关联（多重性）：

- 电梯 1——1 电梯马达；电梯 1——1 超载传感器；电梯 1——1 到达面板；电梯 1——1 目的地面板
- 电梯 1..*——报告 *——到达事件；电梯 1..*——报告 *——目的地事件
- 楼层 1——1..* 召唤面板；楼层 1——到达 *——到达事件
- 召唤面板 1——更新 *——召唤事件
- 电梯事件 1——* 目的地事件；电梯事件 1——* 到达事件

8.6 定义服务及消息连接——服务层 (The Service Layer)

[p.92-95]

8.6.1 服务层的概念

[p.92]

定义服务也即标识对象 & 类的操作。对象 & 类所执行的操作以及对象之间传递的消息反映了对象 & 类的动态关系，也即系统的动态方面。

定义服务的方法：

- 考察系统的责任
- 研究问题域
- 分析对象 & 类状态或追踪对象 & 类的执行线路
- 用例模型中的事件流

8.6.2 定义消息连接

[p.93]

在对象建模中，消息（Message）是对象 & 类之间在行为上的**唯一联系方式**。

定义消息连接的方法：

- 在已经存在实例连接的对象 & 类和分类结构之间增加消息连接
- 检查对象 & 类和分类结构，寻找一个实例所需要的另一个实例的操作

在 UML 中的表示：

- 在类图中，可以采用实例关联的**导航性**来表示消息连接
- 也可采用单独的协作图（Collaboration Diagram）或顺序图（Sequence Diagram）来描述对象 & 类实例的消息连接

8.6.3 电梯控制系统的服务层

[p.94-95]

电梯控制系统的服务层主要依据**事件-响应**模式来说明。

考虑的事件主要有 **7 种**：

1. 召唤电梯
2. 目的地请求
3. 电梯到达调度的楼层
4. 电梯到达非调度的楼层
5. 电梯就绪
6. 电梯超载
7. 电梯未超载

以“召唤电梯”事件为例：

当召唤电梯这一事件识别后，对应的召唤面板要被刷新。当电梯成为“就绪”状态时，电梯调度开始。根据这一事件，须定义如下操作（服务）：

操作 1： 识别召唤请求：“召唤事件”对象 & 类识别召唤电梯事件

操作 2： 报告召唤事件：“召唤事件”对象 & 类向“召唤面板”对象 & 类报告召唤电梯事件的发生，以刷新召唤面板

操作 3： 更新召唤面板：“召唤面板”对象 & 类刷新召唤面板，指示召唤产生

9 需求规格说明与验证 (Requirements Specification & Verification)

9.1 需求规格说明概述

[p.96]

需求分析的主要成果是得到**需求规格说明** (Software Requirements Specifications, SRS)。

需求规格说明的作用：

- 为用户、分析人员、设计人员和测试人员之间的理解和交流提供了方便
- 是系统设计、测试和验收的依据
- 起着控制系统演化过程的作用，追加需求应结合需求规格说明来考虑

9.2 需求规格说明方法

[p.97]

需求规格说明方法分为两大类：

一、 形式化需求规格说明 (Formal Specification)：采用完全精确的**语法和语义**来表达需求。根据表达能力，形式化方法可分为五类：

表 8: 形式化需求规格说明的五种类型

类型	代表方法与说明
基于模型的方法 (Model-Based)	Z 语言： 基于集合论和一阶谓词逻辑，通过状态模式和操作模式来描述系统
代数方法 (Algebraic)	OBJ 语言： 基于代数规约和多类型代数，通过定义数据类型和操作来刻画系统
过程代数方法 (Process Algebra)	CSP (通信顺序进程, Communicating Sequential Processes)：描述并发系统中组件的交互行为
基于逻辑的方法 (Logic-Based)	时态逻辑 (Temporal Logic)：使用逻辑公式描述系统行为随时间变化的性质
基于网络的方法 (Net-Based)	Petri 网 (Petri Net)：使用有向二分图描述系统的并发、异步和分布式行为

一、 非形式化需求规格说明 (Informal Specification)：

- 采用自然语言来描述
- 可以用图表及其它符号帮助理解
- 可用标准化的格式来编制（如 IEEE 830 标准）

9.3 需求验证

[p.98]

9.3.1 需求验证的重要性

大量统计数字表明，软件系统中 **15%** 的错误起源于需求的错误。需求阶段的错误发现得越晚，修复成本越高。

9.3.2 有效需求规格说明的特征

一个有效的需求规格说明应具有如下特征：

- (1) **正确性** (Correctness)：每个需求正确描述了期望的系统能力
- (2) **无歧义性** (Unambiguity)：每个需求只有一种解释
- (3) **完整性** (Completeness)：包含所有重要的需求
- (4) **一致性** (Consistency)：需求之间没有矛盾
- (5) **可验证性** (Verifiability)：存在成本可接受的验证方法
- (6) **可理解性** (Understandability)：客户和开发人员都能理解
- (7) **可修改性** (Modifiability)：结构和风格便于修改
- (8) **可追踪性** (Traceability)：每个需求可以追踪到其来源和对应的设计/实现
- (9) **注释** (Annotated)：包含充分的说明和注释

9.3.3 需求验证完成后的注意事项

一旦需求验证完成，需求规格说明就变成了软件开发的**合同**。尽管允许事后可以修改需求规格说明，但用户应该注意的是，每种事后修改都是对软件范围的扩展，因此可能增加成本和（或）延长软件项目进度。

10 小结 (Summary)

10.1 本章核心要点

[p.99]

- (1) **需求分析的结果**定义了待开发系统的范围，即必须“做什么”，是后续开发活动的基础。
- (2) **需求分析的困难**：需求分析存在许多困难或特点（问题的复杂性、交流障碍、需求易变性、不一致性和不完整性），需要用户和开发人员密切配合，充分交流，借助各种建模方法，详细、准确弄清系统的功能、数据及约束等信息。

(3) 结构化方法的主要特点之一是自顶向下、逐层细化（Top-Down Decomposition），典型代表包括：

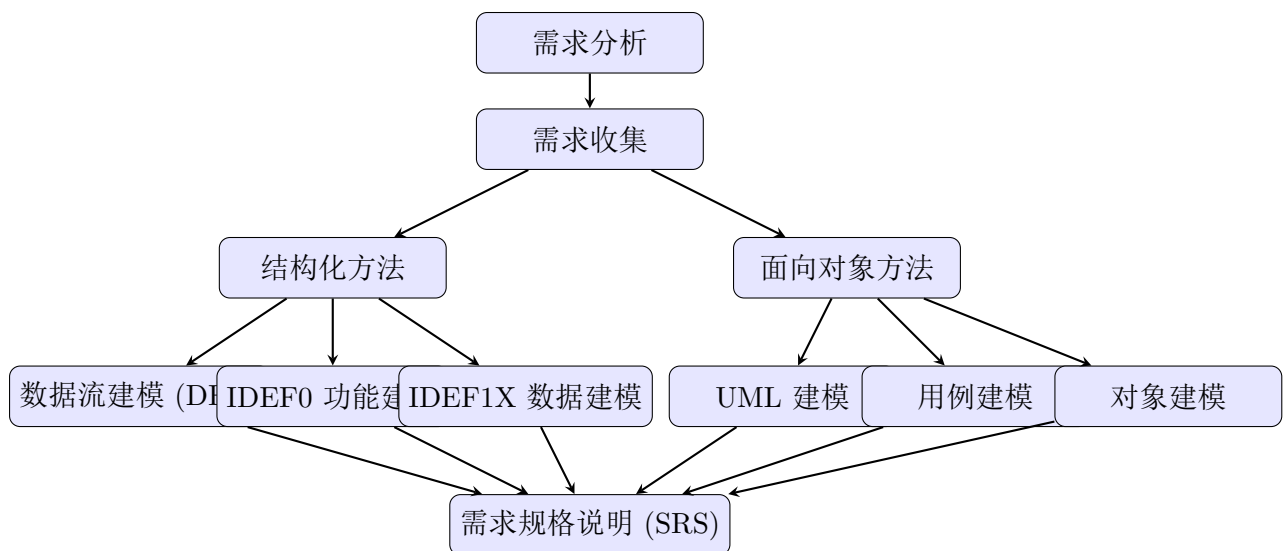
- 数据流建模（DFD）：通过数据流图、数据词典和加工说明描述系统
- IDEF0 功能建模：通过盒子和箭头严格描述系统功能
- IDEF1X 数据建模：通过实体、联系和属性描述数据需求

(4) 面向对象方法则采用一种更接近人类思维方式的形式来刻画实际系统，即对象、类、继承和消息通信。典型代表包括：

- UML 建模语言：提供丰富的图形化建模工具（五大静态图 + 五大动态图）
- 用例建模：从用户视角描述系统功能
- Coad/Yourdon 对象建模：通过五个层次（对象层、结构层、主题层、属性层、服务层）建立对象模型

(5) 需求规格说明：需求分析的最终产物（SRS），是开发合同，需要具备正确性、无歧义性、完整性、一致性等特征。

10.2 本章知识体系图



— 第 5 章完 —